

Назва проекту: Impulse

Вебпосилання на сторінку в GitHub: <https://github.com/yelyzavetaholovko-ui/Impulse>

2. Функціональні вимоги

ID	Функціональна вимога	Пріоритет
FR-01	Система дозволяє вводити стартові показники: вік, зріст, вага, мета, доступний час на тренування.	Високий
FR-02	Система - ШІ-редактор генерує наглядну анімацію майбутнього фізичного вигляду користувача.	Низький
FR-03	Система розраховує норму калорій, генерує план прийомів їжі.	Високий
FR-04	Система формує план тренувань без перевантажень.	Високий
FR-05	Система дозволяє обирати продукти для формування раціону.	Середній
FR-06	Система автоматично відстежує прогрес користувача.	Середній
FR-07	Система автоматично видаляє дані користувача з оперативної системи після завершення ШІ-оброблення.	Високий

3. Діаграма прецедентів

@startuml

left to right direction

actor "Користувач" as User

actor "Зовнішній сервіс харчових даних" as FoodAPI

rectangle "Система планування харчування" {

```

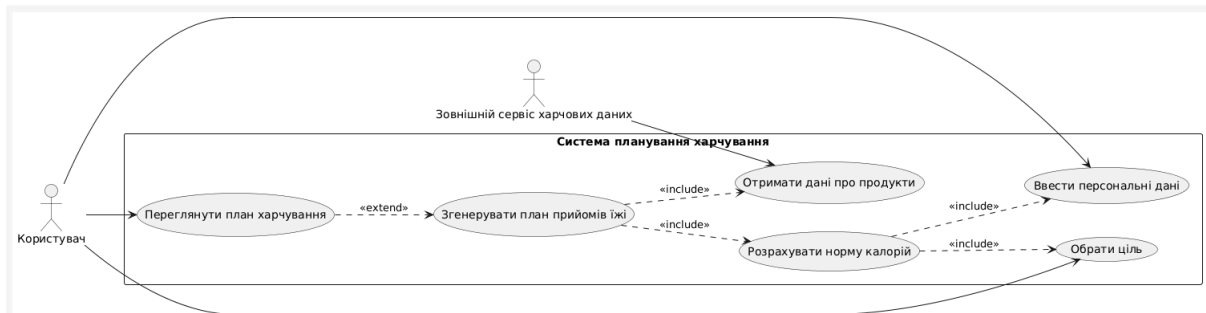
usecase "Ввести персональні дані" as UC01
usecase "Обрати ціль" as UC02
usecase "Розрахувати норму калорій" as UC03
usecase "Згенерувати план прийомів їжі" as UC04
usecase "Переглянути план харчування" as UC05
usecase "Отримати дані про продукти" as UC06
}

```

```

User --> UC01
User --> UC02
User --> UC05
FoodAPI --> UC06
UC03 ..> UC01 : <<include>>
UC03 ..> UC02 : <<include>>
UC04 ..> UC03 : <<include>>
UC04 ..> UC06 : <<include>>
UC05 ..> UC04 : <<extend>>

```



@enduml

4. Діаграма класів

@startuml

```

class User {
    - userId : int
    - name : String
    - age : int
    - weight : float
    - height : float
    - activityLevel : String
}

```

```
+ getProfile() : String
+ updateData(weight : float, height : float) : void
}
```

```
class Goal {
  - type : String
  - calorieAdjustment : float
  + applyAdjustment(baseCalories : float) : float
}
```

```
class CalorieCalculator {
  + calculateCalories(user : User) : float
}
```

```
class MealPlan {
  - planId : int
  - totalCalories : float
  + generate(calories : float) : void
  + getMeals() : List<Meal>
}
```

```
class Meal {
  - name : String
  - calories : float
  + calculateCalories() : float
}
```

```
class Product {
  - name : String
  - caloriesPer100g : float
  + getCalories(amount : float) : float
}
```

```
class FoodService {
  + fetchProduct(name : String) : Product
}
```

```

class PremiumUser {
  - subscriptionLevel : String
  + getAdvancedPlan() : MealPlan
}

```

User "1" -- "1" Goal : has

User ..> CalorieCalculator : uses

User "1" -- "0..*" MealPlan : owns

CalorieCalculator ..> Goal : uses

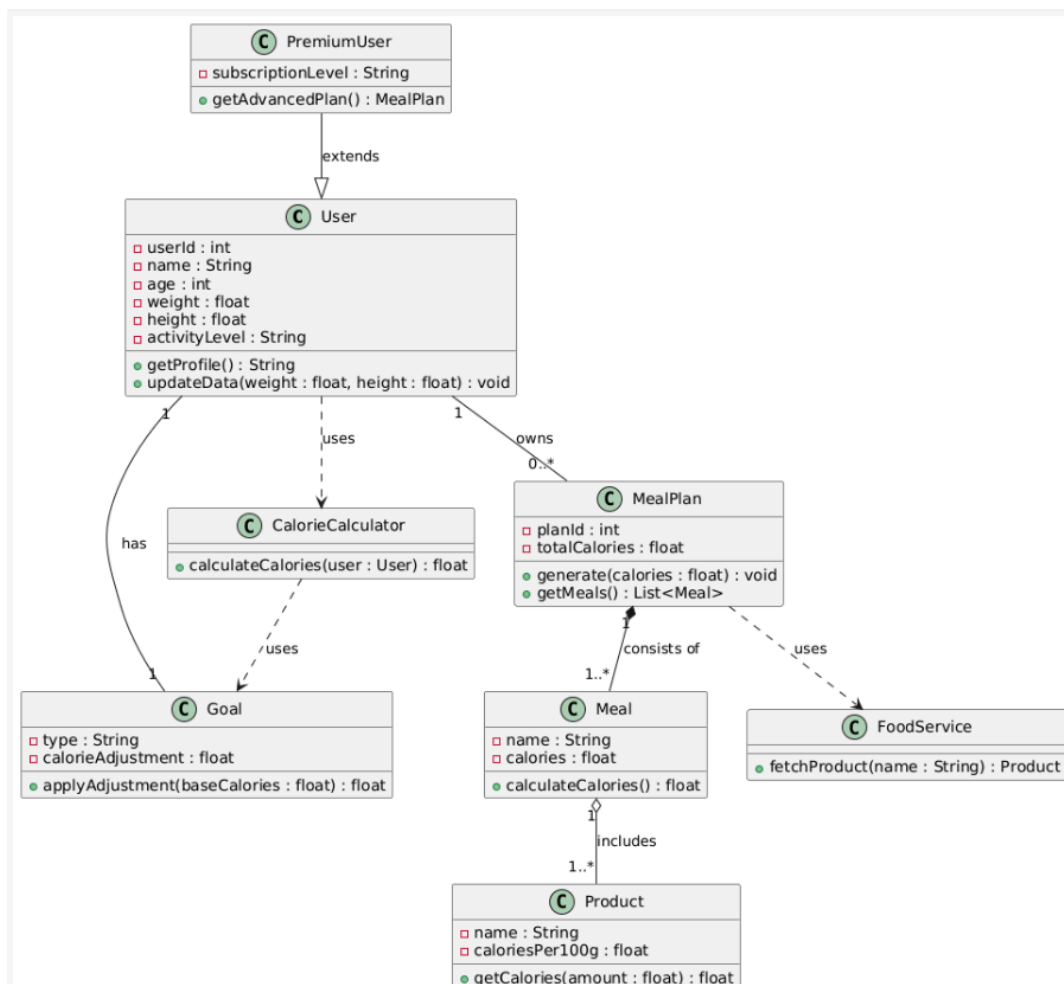
MealPlan "1" *-- "1..*" Meal : consists of

Meal "1" o-- "1..*" Product : includes

MealPlan ..> FoodService : uses

PremiumUser --|> User : extends

@enduml



5. Діаграма послідовності

@startuml

actor User

participant "CalorieCalculator" as CC

participant "Goal" as Goal

participant "FoodService" as FS

participant "MealPlan" as MP

User -> CC : calculateCalories(userData)

activate CC

CC -> Goal : getCalorieAdjustment()

activate Goal

Goal --> CC : adjustment

deactivate Goal

CC --> User : baseCalories

deactivate CC

User -> MP : generatePlan(baseCalories)

activate MP

loop для кожного прийому їжі

MP -> FS : fetchProduct(foodName)

activate FS

FS --> MP : productData

deactivate FS

MP -> MP : calculateMealCalories()

end

MP --> User : mealPlan

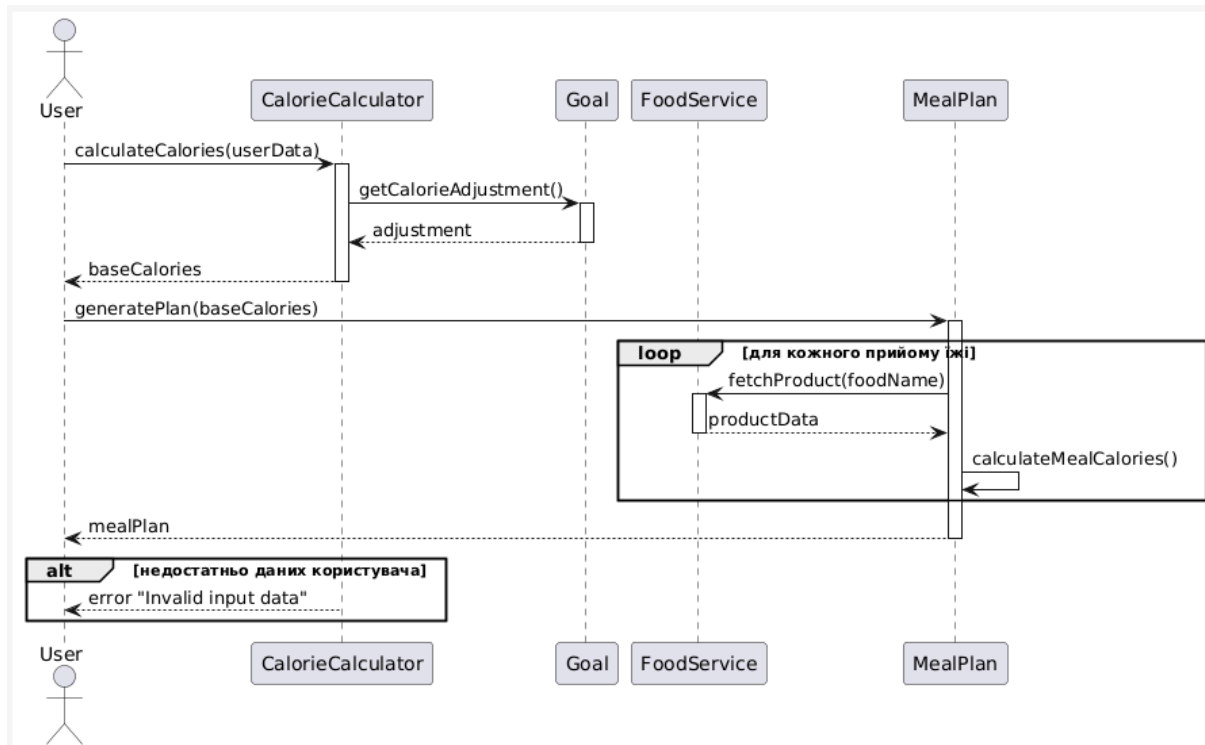
deactivate MP

```

alt недостатньо даних користувача
  CC --> User : error "Invalid input data"
end

```

@enduml



6. Матриця трасовності

Вимога	Use Case	Класи	Sequense
FR-03	UC-03.1: Розрахувати норму калорій UC-03.2: Згенерувати план прийомів їжі	User Goal CalorieCalculator MealPlan Meal Product FoodService	SD-03: "Отримання персонального плану харчування"